Calibration de la caméra principale d'un iPhone 15 Pro

1 Objectif et modèle

Le but de ce TP est de mesurer les paramètres internes de la caméra de mon téléphone, en traitant le capteur comme l'objet d'une expérience. Je suis les tutoriels OpenCV « Camera Calibration » et « Pose Estimation » en Python, avec OpenCV 5.0.

J'utilise le modèle sténopé vu en cours. Un point (X,Y,Z) du repère de la mire se projette en un pixel (u,v) par

$$s \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = K \, [R \,|\, t] \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}, \qquad K = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}.$$

La matrice K regroupe les paramètres intrinsèques : les focales f_x, f_y exprimées en pixels et le point principal (c_x,c_y). La rotation R et la translation t sont les paramètres extrinsèques, propres à chaque photo. OpenCV ajoute un modèle de distorsion à cinq coefficients, trois radiaux (k_1,k_2,k_3) et deux tangentiels (p_1,p_2). La fonction `calibrateCamera` estime tout cela à partir de plusieurs vues d'une mire plane, selon la méthode de Zhang [1].

2 Protocole expérimental

Le capteur. La caméra étudiée est la caméra principale (« grand-angle ») de l'iPhone 15 Pro. D'après les métadonnées EXIF des photos, l'objectif a une focale nominale de 6,765 mm, ouvert à $f/1{,}78$, soit 24 mm en équivalent 24×36. Le capteur fait 48 Mpx (8064×6048 photosites de 1,22 μm). Les photos que j'ai utilisées sont en 24 Mpx, au format portrait : 4284×5712 pixels. Avec les dimensions du capteur, cela donne une zone sensible de 7,379×9,838 mm en portrait, valeurs que je passe à `calibrationMatrixValues`.

La mire. J'ai affiché un damier sur l'écran de mon ordinateur portable. Il compte 10×7 cases, soit 9×6 coins internes. J'ai mesuré la taille d'une case à la règle sur l'écran : 19 mm. Un écran a l'avantage d'être parfaitement plan et rigide, mais il produit des reflets et du moiré quand on le photographie de près.

Les prises de vue. J'ai pris 15 photos le 4 novembre 2026, toutes en deux minutes environ, en faisant varier la distance et l'inclinaison du téléphone. L'iPhone enregistre en HEIC ; un petit script (`convertir.py`) les convertit en JPEG en conservant les EXIF. La mise au point et l'exposition sont restées automatiques (1/50 ou 1/100 s, ISO 100 à 200).

3 Chaîne de traitement

Le code et les 15 photos sont disponibles sur GitHub :

`https://github.com/Harena21/calibration-camera-ticv2026`

Le script `calibration_Harena.py` suit le tutoriel OpenCV, avec deux adaptations liées à la taille des images. `findChessboardCorners` est lent sur une image de 24 Mpx. Je cherche donc le

damier sur une copie réduite au quart, puis je remets les coordonnées à l'échelle et je les affine au sous-pixel avec `cornerSubPix` sur l'image pleine résolution, avec une fenêtre de 21×21 pixels.

`calibrateCamera` donne ensuite K, les coefficients de distorsion et une pose par image. Je calcule aussi l'erreur de reprojection photo par photo, pour repérer les mauvaises vues. La fonction `calibrationMatrixValues` convertit ensuite K en champs de vision, en focale en millimètres et en position du point principal sur le capteur.

Pour finir, une image est corrigée avec `undistort`. Sur trois photos, `solvePnPRansac` estime la pose du damier, puis `projectPoints` dessine les axes et un cube posé sur la mire.

4 Résultats

Erreur par image. Le damier est détecté sur les 15 photos, et je les utilise toutes pour la calibration. Le tableau 1 donne l'erreur de reprojection de chaque image. Treize photos restent entre 0,6 et 1,1 px et IMG_3279 monte à 1,23 px, mais une photo se détache nettement : IMG_3280, avec 3,14 px. En la regardant (figure 1a), je vois que c'est la vue la plus rasante de la série. Les cases du fond y sont floues, car la profondeur de champ est faible à $f/1{,}78$, et le moiré de l'écran est bien visible. J'ai vérifié que ce n'est pas un simple flou de bougé : la variance du laplacien de cette image est du même ordre que celle d'autres photos bien reprojetées.

Image	3268	3269	3270	3271	3272	3273	3274	3275
Erreur (px)	0,89	0,76	0,88	0,71	0,81	0,84	1,06	0,91
Image	3276	3277	3278	3279	3280	3281	3283	
Erreur (px)	0,67	0,67	0,97	1,23	3,14	0,82	0,59	

TABLE 1 – Erreur de reprojection (RMS) par image, calibration sur les 15 photos.

Paramètres intrinsèques. Avec les 15 images, l'erreur RMS globale vaut 1,16 px et on obtient

$$K = \begin{pmatrix} 4108{,}3 & 0 & 2121{,}5 \\ 0 & 4112{,}9 & 2873{,}9 \\ 0 & 0 & 1 \end{pmatrix},$$

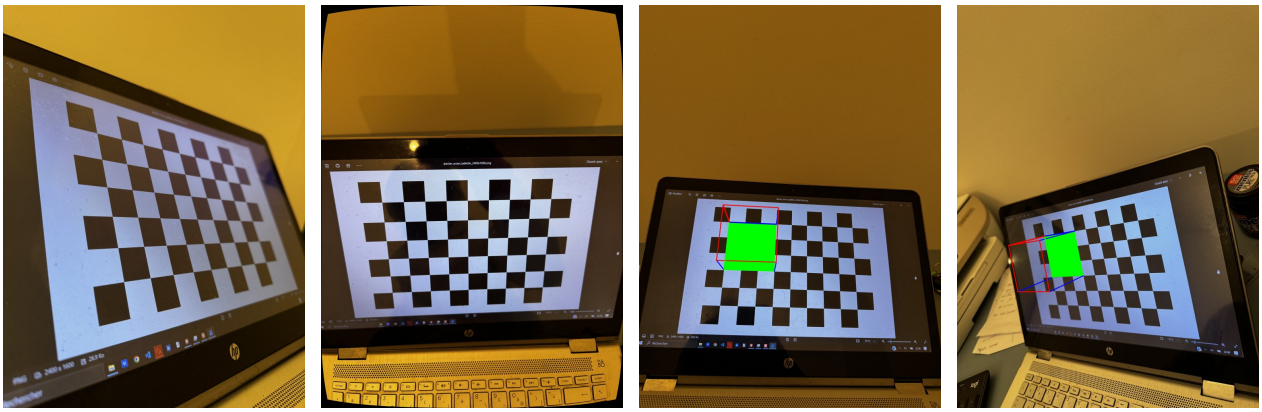
$$(k_1,k_2,p_1,p_2,k_3) = (0{,}238;\ -0{,}946;\ -0{,}0002;\ -0{,}0017;\ 1{,}097).$$

Les deux focales en pixels diffèrent de 0,1 % (rapport d'aspect 1,0011), ce qui correspond bien à des pixels carrés. Le point principal est décalé d'environ 21 px vers la gauche et 18 px vers le bas par rapport au centre de l'image (2142, 2856), soit moins de 0,5 % de la taille de l'image.

Focale et champ de vision. `calibrationMatrixValues` donne un champ horizontal de 55,1° et un champ vertical de 69,6° (en portrait). La focale vaut

$$f = 7{,}08 \text{ mm} = 0{,}00708 \text{ m}.$$

Essai sans IMG_3280. Par curiosité, j'ai relancé la calibration en retirant cette photo. L'erreur RMS descend à 0,85 px, soit un quart de moins, alors que les paramètres bougent peu : f_x passe de 4108 à 4094 px et la focale de 7,08 à 7,05 mm. Une seule vue mal reprojetée suffit donc à gonfler l'erreur moyenne, sans beaucoup déplacer K. Je garde quand même les 15 photos dans les résultats de ce rapport, en notant IMG_3280 comme la vue la moins fiable.



(a) IMG_3280 (b) Image corrigée (c) Pose, IMG_3268 (d) Pose, IMG_3283

FIGURE 1 – (a) La vue rasante, la moins bien reprojetée. (b) Image IMG_3269 après correction de la distorsion. (c, d) Cube de 3 cases de côté projeté avec la pose estimée.

Correction et pose. La figure 1b montre une image corrigée de sa distorsion. J'ai gardé tous les pixels de l'image d'origine (paramètre $\alpha = 1$ de `getOptimalNewCameraMatrix`), d'où les bords noirs courbés. Les figures 1c et d montrent le cube projeté sur deux photos : sa base reste alignée sur les cases, y compris sur la vue inclinée de la figure 1d. `solvePnPRansac` donne des distances caméra-damier de 36,2 cm, 31,9 cm et 41,4 cm pour les trois photos traitées.

5 Discussion

Écart avec la focale constructeur. Ma mesure (7,08 mm) dépasse d'environ 5 % la focale indiquée dans les EXIF (6,765 mm). Je vois trois causes possibles. D'abord, la focale EXIF est la focale nominale, mise au point à l'infini. Or la calibration mesure la distance entre le centre optique et le capteur au moment de la prise de vue, et la mise au point automatique se faisait sur le damier, situé entre 32 et 41 cm. La formule de conjugaison donne alors $d' = f\,d/(d-f) = 6{,}765 \times 362/(362-6{,}765) \approx 6{,}89$ mm, ce qui explique environ 40 % de l'écart. Ensuite, la conversion en millimètres repose sur la taille du capteur que j'ai supposée : une erreur de 1 % sur la taille des photosites donne 1 % d'erreur sur f, alors que f_x en pixels n'en dépend pas. Enfin, l'image que je calibre n'est pas celle du capteur. Le passage de 8064 à 5712 pixels de haut ne correspond pas à un regroupement entier de photosites, donc l'image est rééchantillonnée, et le traitement d'Apple corrige probablement déjà une partie de la distorsion. Je ne peux pas le vérifier sans photo RAW.

Distorsion. Les coefficients radiaux sont grands et de signes alternés ($k_1 > 0$, $k_2 < 0$, $k_3 > 0$). Pris séparément, ils n'ont pas beaucoup de sens : ils se compensent en partie, et ils sont mal contraints parce que le damier n'occupe jamais les coins de l'image. Pour bien estimer la distorsion en bord de champ, il faudrait des vues où la mire touche les bords.

Taille de la case. La taille de la case (19 mm, mesurée à la règle) n'intervient pas dans K, puisque changer l'échelle de la mire change seulement les translations. En revanche, elle se reporte directement sur les distances : une erreur de 0,5 mm sur la case donne environ 2,6 % d'erreur, soit à peu près 1 cm à 36 cm.

Pose à partir de points coplanaires. Les 54 coins du damier sont coplanaires. L'article de Quan et Lan [2] montre qu'à partir de quatre points, la pose est déterminée de façon unique hors configurations critiques, et que leurs méthodes linéaires ne dégénèrent pas dans le cas coplanaire.

Avec 54 points et un tirage RANSAC (Fischler et Bolles, cité dans le même article), `solvePnPRansac` dispose de bien plus de points que le minimum, et le cube projeté tombe correctement sur la mire.

Résolution et qualité. Les 24 Mpx ne donnent pas 24 millions de détails indépendants. À $f/1{,}78$, la tache de diffraction d'Airy a un diamètre d'environ $2{,}44\,\lambda N \approx 2{,}4$ μm pour $\lambda = 550$ nm, alors qu'un pixel de l'image finale correspond à 1,72 μm sur le capteur. L'optique étale donc déjà un point sur plus d'un pixel, avant même le traitement et la compression. Une erreur de 1,16 px équivaut à environ 2 μm sur le capteur (1,5 μm sans IMG_3280), du même ordre que cette limite.

6 Limites et améliorations

Avec plus de temps, je referais l'expérience avec une mire imprimée et collée sur un support rigide pour éviter le moiré, en bloquant la mise au point (verrouillage AE/AF) pour garder une focale constante. Je prendrais aussi plus de vues où la mire couvre les bords et les coins de l'image, et des photos RAW pour échapper au traitement d'Apple. Une validation croisée, qui recalibre en retirant une image à la fois comme je l'ai fait pour IMG_3280, donnerait une incertitude sur f au lieu d'une seule valeur.

Références

[1] Z. Zhang, « A flexible new technique for camera calibration », *IEEE Trans. Pattern Analysis and Machine Intelligence*, 22(11), 2000.

[2] L. Quan et Z. Lan, « Linear N-point camera pose determination », *IEEE Trans. Pattern Analysis and Machine Intelligence*, 21(7), 1999.

[3] OpenCV, tutoriels « Camera Calibration » et « Pose Estimation », `https://docs.opencv.org/4.x/d4/d94/tutorial_camera_calibration.html` et `https://docs.opencv.org/3.4/d7/d53/tutorial_py_pose.html`.